Quantum Linear System Algorithms for Graph-Based Financial Risk Diffusion

Group: 26A
Team Members: Harris Ahmed M, Neha Girdhar, Snigdha Chandan, Selvakumar Arumugam, Rishanthi Ramakrishnan

July 7, 2026

1 Introduction

Quantum Linear System Algorithms (QLSAs) are among the most important applications of quantum computing because they provide the computational primitive underlying numerous quantum algorithms for scientific computing, optimization, machine learning, and network analysis. In this project, we investigated the application of QLSAs to graph-based financial risk diffusion, where systemic risk propagates through a sparse financial network and the resulting risk scores are obtained by solving sparse linear systems.

Rather than studying only the textbook HHL algorithm, we explored the complete algorithmic pipeline required for practical quantum linear-system solvers. This included mathematical modeling of graph diffusion, sparse state preparation, Hamiltonian simulation, block encoding, eigenvalue inversion using both QPE-based HHL and modern QSVT techniques, and efficient readout strategies for extracting useful financial observables. We also compared multiple implementation architectures, analyzed their resource requirements and asymptotic complexity, and briefly contrasted them with the best-known classical approaches.

2 Problem Statement

In a nutshell, we studied the sparse systems:

$$[\alpha L + (1-\alpha)I]\, x = (1-\alpha)c$$

and

$$(I - \alpha P_{\text{sym}})x = (1-\alpha)c$$

arising from graph-Laplacian diffusion in financial networks. The objective is the rapid estimation of financial risk metrics, such as identifying the top-k riskiest institutions and estimating the risk score of a specified institution.

2.1 Motivation

Suppose we have N banks that are sparsely connected to each other in a huge network. Suppose we know that a few banks are known to have a sudden increase in risk scores. A natural question arises. What other banks are influenced by this event and by how much? This problem can be formulated as a graph-diffusion based *semi-supervised* learning problem (Zhu et al. 2003 [ZGL03], Zhou et al. 2004[ZBL$^+$04], Belkin et al. 2004 [BMN04]). We can think of risk as

a contagion propagating across the network, with the strength of propagation measured by a parameter α. Then the risk score (a nonnegative quantity) for each bank can be thought of as labels to be predicted. A few known banks with high risk scores are thought of as initial shocks that drive the diffusion process. The problem then is to predict the near-term steady state risk scores of all banks in the network. Intuitively, those banks which are strongly connected to risky banks tend to be risky as well. So, the strength of connection between any two banks is a key factor that drives the diffusion. Figure 1 shows a simple 5-node example. Figure 2 demonstrates how risk propagates in a network. The weights of the edges in the graph represent connection strengths. We consider only undirected graphs in this project, so the corresponding adjacency matrix J is symmetric.

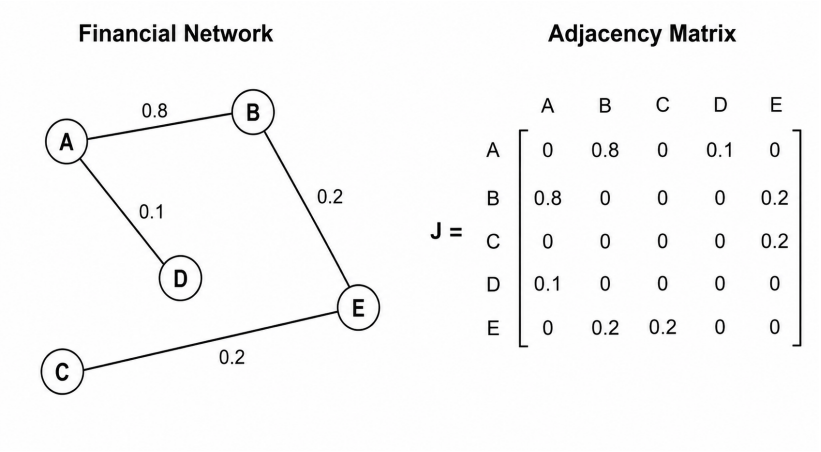

Figure 1: Example weighted financial network (left) and its corresponding adjacency matrix J (right).

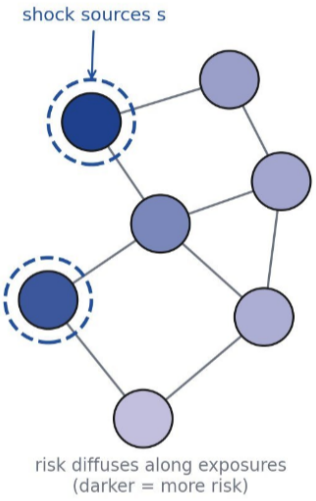

Figure 2: Illustration of financial risk diffusion.

2.2 The Model

Let

Graph $G = (V, E)$	: financial network
N	: the number of nodes (vertices) in the graph
J	: adjacency matrix, where J_{ij} represents risk exposure from node i to j
	:(where we assume $J_{ij} \in [0, 1]$, $J_{ii} = 0$ for all i, and $J = J^T$)
s_J	: sparsity of J - the maximum number of nonzero entries in any row of J
D	: diagonal degree matrix, where $D_{ii} = \sum_j J_{ij}$
	:(we assume each node is connected to at least one other node)
	:(so $D_{ii} > 0$ for all i)
$L := D - J$	: graph Laplacian
$P := D^{-1}J$	: transition matrix (row-normalized adjacency matrix)
$P_{\text{sym}} := D^{-1/2}JD^{-1/2}$	: symmetrically normalized adjacency matrix
$\alpha \in (0, 1)$	: diffusion parameter controlling propagation strength
c	: initial risk vector (shock vector) (e.g., distressed institutions)
x	: propagated systemic risk scores

The basic model is derived using the graph Laplacian operator L that drives smooth propagation across connected institutions. Our goal is to find a solution x that is as close to the initial shock vector c as possible while also such that strongly connected nodes share similar risk scores. These two conditions prioritized by α are precisely captured by the minimization problem:

$$\min_x \quad \left(\frac{\alpha}{2}\right) x^T L x + \left(\frac{1-\alpha}{2}\right) \|x - c\|_2^2.$$

The term $x^T Lx$ is the graph Dirichlet energy which penalizes assigning very different risk scores to strongly connected institutions. Taking the gradient and setting it to zero yields the linear equation

$$[\alpha L + (1-\alpha)I]\, x = (1-\alpha)c \qquad \text{(Model 1)}$$

Solving this equation gives us the risk scores x. However, this model is biased towards nodes that are strongly connected to many nodes. A more realistic model is obtained when we normalize the Laplacian so that the bias is removed. This model is given by

$$\left[\alpha D^{-1}L + (1-\alpha)I\right] x = (1-\alpha)c \qquad \text{(Model 3)}$$

which is the same as

$$(I - \alpha P)x = (1-\alpha)c$$

where $P = D^{-1}J$ defines the transition matrix for a random walk by the contagion. However, with the above model the matrix $(I - \alpha P)$ is not generally Hermitian and can be poorly conditioned. Although a HHL-based quantum linear solver can still solve this, this formulation is less suitable for HHL because the numerical conditioning of $(I - \alpha P)$ is governed by singular values rather than eigenvalues. So we chose a nice middle ground model created using a *symmetrically* normalized Laplacian $L_{\text{sym}} := D^{-1/2}LD^{-1/2}$:

$$[\alpha L_{\text{sym}} + (1-\alpha)I]\, x = (1-\alpha)c \qquad \text{(Model 2)}$$

which can be rewritten as

$$(I - \alpha P_{\text{sym}})x = (1 - \alpha)c$$

where $P_{\text{sym}} = D^{-1/2} J D^{-1/2}$. This time the matrix $(I - \alpha P_{\text{sym}})$ is both Hermitian and well-conditioned for realistic values of α, making it ideal for HHL-based quantum linear solver algorithms to be exploited.

Let A denote the matrix to be inverted (in all three models). It is easily seen that L is positive semidefinite. The spectral analysis in (subsection 3.2) immediately shows that A is positive definite for both Model 1 and Model 2. Among the three formulations, we focus primarily on Model 2 because it is Hermitian, positive definite, and admits a degree-independent bound on the condition number (subsection 3.2), making it particularly well suited to HHL- and QSVT-based quantum linear-system algorithms.

2.3 Quantum Formulation and Solution

We do not seek the full solution x. We consider only two realistic use cases.

(i) What are the top-k risky nodes?

(ii) What is the estimated risk score for a particular node?

Fortunately, these questions can be answered without having to solve for x completely. It suffices to find a normalized solution state $|x\rangle$ proportional to the true solution. From this state, the most probable computational basis states can be found to answer (i) and a simple measurement circuit realizing the observable $|\eta\rangle\langle\eta|$ can answer (ii) for a node indexed by $|\eta\rangle$. Additionally, we make a reasonable realistic assumption. The initial risk (shock) vector c is always of the form of a vector proportional to a sparse equal superposition state $|b\rangle$, that is, of the form where

$$|b\rangle = \sum_{j=0}^{\gamma-1} \frac{1}{\sqrt{\gamma}} |i_j\rangle$$

where $\{i_0, i_1, \cdots, i_{\gamma-1}\}$ are the indices of the shocked institutions, and the maximum number of nonzero computational basis components γ in the state $|b\rangle$ satisfies $\gamma = O(\log N)$ for realistic use cases. Our problem now is to solve:

$$A|x\rangle = |b\rangle .$$

Here are some additional variables of importance for our quantum approach to the solution.

κ	: condition number of the matrix A
s	: sparsity of A - the maximum number of nonzero entries in any row of A
	: (Note: $s = s_J + 1$, and we assume $s = O(\log N)$)

3 Work Done

We analyzed the problem mathematically, investigated sparse state preparation, Hamiltonian simulation, block encoding, eigenvalue inversion (HHL and QSVT), and efficient measurement strategies. Multiple HHL variants were implemented and validated against classical solutions.

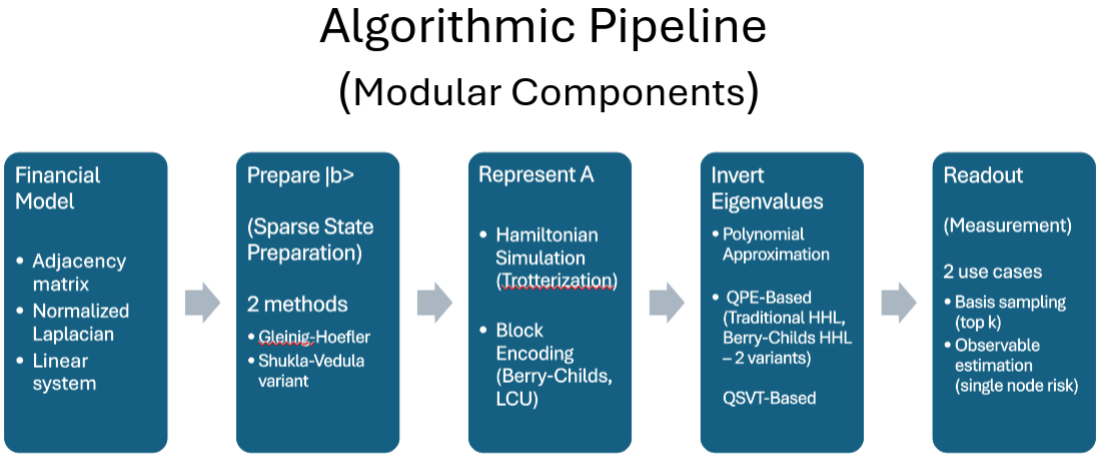

Figure 3: Flow diagram describing the various approaches explored in different modular components of the algorithmic pipeline

3.1 Implementation Evolution

After verifying textbook HHL algorithms for tiny matrices, we explored multiple implementation architectures. Figure 3 shows a high level overview of the algorithmic pipeline leading to the solution, along with the different approaches we explored.

Motivated by the evolution of quantum linear solvers, we decided to explore the following architectures.

1. Traditional HHL based on Hamiltonian simulation through decomposition into $1-$sparse matrices followed by Trotterization and QPE (Harrow, Hassidim, and Lloyd - 2009 [HHL09]).

2. A hybrid approach involving the quantum-walk-based block encoding of the matrix A defined by Berry-Childs [BC12], followed by traditional QPE. Following QPE, we also explored and implemented two variants in the architecture for eigenvalue inversion.

   (a) **Variant 1 (Compute, then rotate)**: Basis-encode the polynomial approximation $P(\nu) \approx \arcsin\left(\frac{C}{\alpha_A \cos(2\pi\nu)}\right)$ in an auxiliary register via a reversible polynomial-evaluation oracle, prepare the HHL ancilla amplitude using controlled rotations driven by the basis-encoded polynomial value, and finally uncompute the auxiliary register.

   (b) **Variant 2 (Rotate while computing)**: Compile the polynomial coefficients in the Boolean monomial basis directly into parameterized single-qubit rotations, enabling coherent evaluation of $P(\nu)$ within the circuit and direct preparation of the HHL ancilla amplitude, without explicitly basis-encoding the polynomial value.

3. A modern quantum linear solver based on first decomposing A into a linear combination of unitaries (LCU) (Childs and Wiebe. 2012 [CW12], Berry et al. 2015[BCC$^+$15]), and then block encoding it in a quantum walk through the PREPARE-SELECT-PREPARE† architecture (Low and Chuang. 2016 [LC16]), followed by Quantum Singular Value Transormation (QSVT) (Gilyen et al. 2019 [GSLW19], Low and Chuang. 2019[LC19]) to get the solution.

3.2 Spectral Properties of A

We analyzed the spectral properties of A to begin with. Let us consider the three cases:

1. **Model 1** (Unnormalized Laplacian): Here, $A = [\alpha L + (1-\alpha)I]$ is Hermitian positive definite because the graph Laplacian L is symmetric positive semidefinite. Its eigenvalues satisfy
$$1 - \alpha \leq \lambda(A) \leq 1 - \alpha + 2\alpha d_{\max}$$
where $d_{\max}$ is the maximum node degree (maximum entry in D). So, the condition number may grow with the maximum node degree.

2. **Model 2** (Symmetrically Normalized Laplacian): Here, $A = [\alpha L_{\text{sym}} + (1-\alpha)I]$ which is $(I - \alpha P_{\text{sym}})$, also Hermitian. Since P_{sym} is both Hermitian and similar to P which is a row stochastic matrix, its eigenvalues lie in $[-1, 1]$. From this, we immediately deduce
$$1 - \alpha \leq \lambda(A) \leq 1 + \alpha$$
and we now have a degree-independent condition number bound: $\kappa(A) \leq \frac{1+\alpha}{1-\alpha}$.

3. **Model 3** (Normalized Laplacian): Here, $A = \left[\alpha D^{-1}L + (1-\alpha)I\right]$ which is $(I - \alpha P)$. While the eigenvalues of P lie in $[-1, 1]$ (as noted above), P is not Hermitian in general. Consequently, its numerical conditioning is governed by singular values rather than eigenvalues, leading only to weaker bounds on singular values $\sigma(A)$ such as
$$1 - \alpha\sqrt{s} \leq \sigma(A) \leq 1 + \alpha\sqrt{s}$$
when $\alpha < \frac{1}{\sqrt{s}}$. This makes Model 2 the most attractive formulation for HHL- and QSVT-based quantum linear-system algorithms.

3.3 Sparse State Preparation - Preparing $|b\rangle$

We independently implemented Gleinig-Hoefler [GH21] state preparation and a variant of Shukla-Vedula [SV24] contiguous state preparation for the sparse state preparation of $|b\rangle$. Both can prepare the state in $O((\log N)^2)$ time, as we assumed $\gamma = O(\log N)$. The latter method can be used without the need for controlled rotation gates and complex circuitry at the cost of a few dummy nodes $O(\gamma)$ in the system. Since our shock vector is a sparse equal-superposition state, the latter is particularly well suited to our problem. Figure 4 shows a simple circuit that prepares the sparse state
$$|b\rangle = \frac{1}{2}\left(|1\rangle + |3\rangle + |21\rangle + |28\rangle\right)$$
in the 5-qubit system register

3.4 Hamiltonian Simulation and Block Encoding - Representing A

We explored 3 types of representing the matrix A, one is through Hamiltonian simulation through sparse decomposition and Trotterization. The other 2 types are based on block encoding.

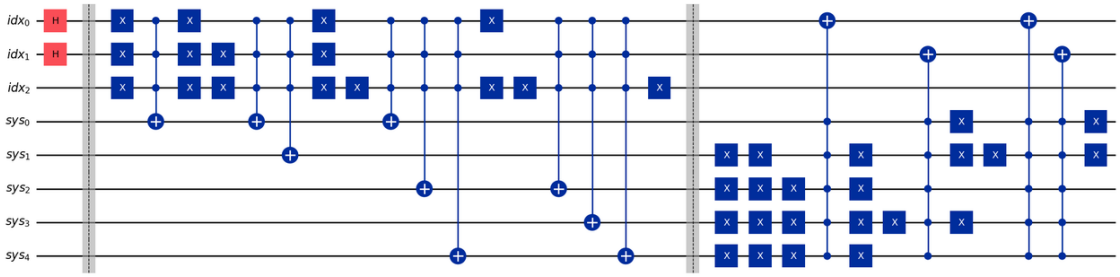

Figure 4: A circuit that prepares the state $|b\rangle = \frac{1}{2}(|1\rangle + |3\rangle + |21\rangle + |28\rangle)$

3.4.1 Hamiltonian simulation through sparse decomposition and Trotterization

While we explored Hamiltonian simulation through decomposition of the matrix A into 1-sparse matrices followed by Trotterization, we skipped implementing this as we moved in favor of the modern block encoding techniques. We observed that decomposing our s-sparse matrices into 1-sparse matrices takes $O(N)$ time as the problem is equivalent to an edge-coloring problem. Additionally, a good performance with Trotterization requires extremely short time steps, and this adversely affects complexity (Berry and Childs. 2012[BC12]).

3.4.2 Block encoding

Block encoding and qubitization (Low and Chuang. 2016 [LC16]) introduced the modern abstraction of embedding a non-unitary matrix A/α_A as the principal block of a unitary matrix U_A, and showed evolution-derived encodings present each eigenvalue of the encoded matrix as a two-dimensional invariant subspace on which a walk operator acts as a rotation. Figure 5 illustrates block encoding. This block encoding becomes the fundamental primitive consumed by both the Berry–Childs HHL variants and the LCU-QSVT solver.

We implemented two different types of block encoding.

1. **Block Encoding using Berry-Childs quantum-walk operator:** We encoded A using the Berry-Childs quantum-walk operator W that is defined as the composition of a swap operator and a reflection operator. The reflection operator reflects the input state about the image space of another operator T, called the Berry-Childs isometry.

   Here is the process we followed.

   (i) We constructed sparse oracles for the Berry-Childs state preparation algorithm to access the positions and the values of the entries of A.

   (ii) We encoded entries of A into the Berry-Childs state. We carefully engineered the failure basis indices so that those basis states are truly orthogonal to the signal space. We also exploited the structure in our matrix A to efficiently encode the sign of matrix entries.

   (iii) We implemented the quantum walk algorithmically through compiler-generated local unitary synthesis and sliced unitary execution.

2. **Block encoding using Linear Combination of Unitaries (LCUs):** The LCU framework, introduced by Childs and Wiebe ([CW12], and subsequently generalized through

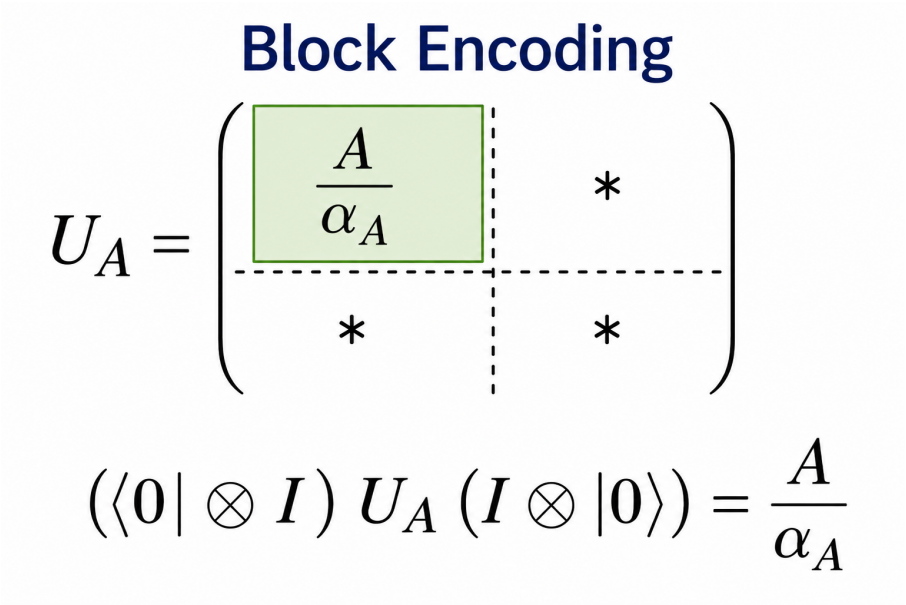

Figure 5: Illustration of a block encoding of the normalized system matrix A. The unitary U_A embeds the scaled matrix A/α_A in its upper-left block, while the remaining blocks are unconstrained. Projecting the ancilla register onto the $|0\rangle$ state recovers the encoded matrix.

qubitization [LC19]), constructs a block encoding by first expressing the normalized system matrix as a linear combination of efficiently implementable unitary operators,

$$A = \sum_i a_i U_i.$$

A PREPARE circuit initializes an ancilla state whose amplitudes encode the normalized coefficients a_i/α_A, while a SELECT applies the corresponding unitary U_i conditioned on the ancilla state. Together these operations realize a unitary U_A satisfying

$$(\langle 0| \otimes I)U_A(I \otimes |0\rangle) = \frac{A}{\alpha_A}$$

thereby providing the block encoding required by the QSVT algorithm.

In the generic LCU framework, one constructs a block encoding by expressing a target operator M as a linear combination of efficiently implementable unitaries,

$$M = \sum_i a_i U_i.$$

PREPARE and SELECT then realize a block encoding of M. Our implementation specializes this idea by choosing $M = \cos(tA)$ and using

$$\cos(tA) = \frac{1}{2}\left(e^{itA} + e^{-itA}\right)$$

where the Hamiltonian evolution operators $e^{\pm iAt}$ are the unitaries appearing in the LCU decomposition. Note that this is in contrast to the direct LCU decomposition of the matrix A itself. The advantage of this decomposition is that it contains only two equally weighted unitary terms, allowing the PREPARE–SELECT construction to be implemented using only a single ancilla qubit, unlike generic LCU decompositions involving many unitary terms. The operator $\cos(tA)$ serves as the signal operator for the subsequent QSVT polynomial transformation.

3.5 Inverting Eigenvalues

We implemented two QPE-based approaches and one QSVT-based approach. In our implementations, all eigenvalue-inversion routines were based on Chebyshev polynomial approximations to approximate either a scaled inverse function $f(x) = \frac{C}{x}$ or a function composed of a scaled inverse function such as $g(x) = \arcsin\left(\frac{C}{\alpha_A \cos(2\pi x)}\right)$.

3.5.1 QPE-based

As discussed earlier near the algorithmic pipeline, we implemented 2 different architectures for QPE-based eigenvalue inversion. Figure 6 and Figure 7 show the difference in architecture.

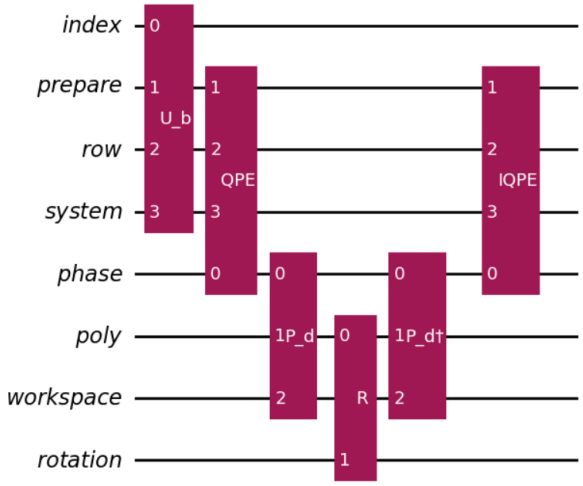

Figure 6: Eigenvalue Inversion Architecture - Variant 1 (Compute, then rotate)

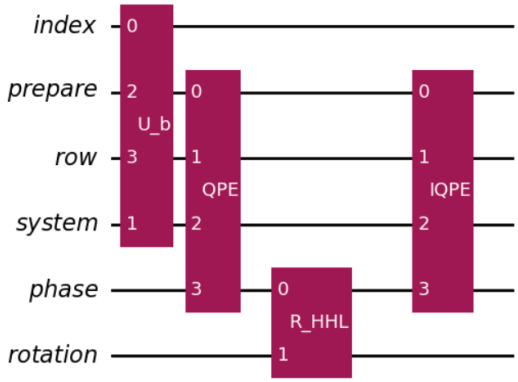

Figure 7: Eigenvalue Inversion Architecture - Variant 2 (Rotate while computing)

1. **Variant 1 (Compute, then rotate)**: Here, we basis-encode the polynomial approximation $P(\nu) \approx \arcsin\left(\frac{C}{\alpha_A \cos(2\pi\nu)}\right)$ in an auxiliary register ("poly" in Figure 6) via a reversible polynomial-evaluation oracle, prepare the HHL ancilla amplitude using controlled rotations driven by the basis-encoded polynomial value, and finally uncompute the auxiliary register. This architecture thus requires more qubits and gates to implement eigenvalue inversion. However, the circuit for preparing the desired ancilla state following eigenvalue inversion is very simple Figure 8.

2. **Variant 2 (Rotate while computing)**: Here, we compile the polynomial coefficients in the Boolean monomial basis directly into parameterized single-qubit rotations, enabling coherent evaluation of $P(\nu)$ within the circuit and direct preparation of the HHL ancilla amplitude, without explicitly basis-encoding the polynomial value. Here we save on the extra qubits needed for computing and storing the inverse eigenvalues. However, we need a very deep circuit (Figure 9) involving an exponential number of multicontrolled rotation gates to accomplish this. Maintaining noise-free coherence is therefore hard.

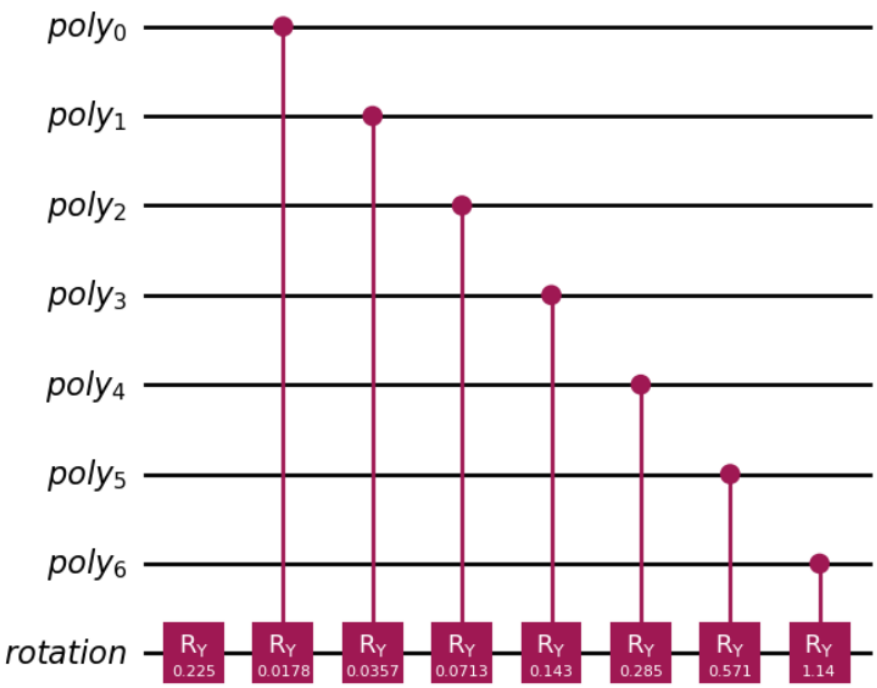

Figure 8: Variant 1 Circuit that encodes the ancilla qubit in the "rotation" register

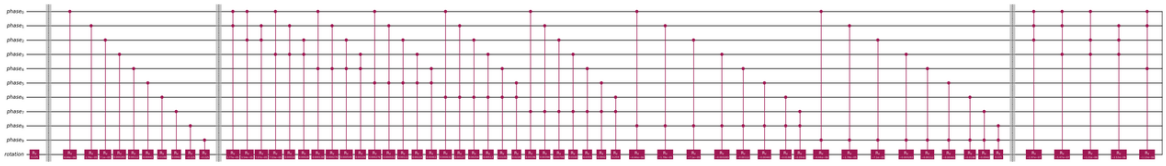

Figure 9: Beginning part of Variant 2 Circuit that encodes the ancilla qubit in the "rotation" register

3.5.2 QSVT-based

Quantum Singular Value Transformation (QSVT) provides a direct method for applying polynomial transformations to the singular values (or eigenvalues, since A is Hermitian) of a block-encoded matrix. Unlike HHL, QSVT does not require Quantum Phase Estimation, an eigenvalue register, or controlled rotation gates. Instead, a sequence of single-qubit phase rotations is interleaved with repeated applications of the block encoding to synthesize a polynomial approximation to the desired matrix function. For solving the linear system, a Chebyshev polynomial approximating the inverse function

$$f(x) = \frac{C}{x}$$

over the spectral interval of the normalized matrix was constructed subject to the boundedness constraints required by QSVT. The resulting phase sequence was then used to implement the polynomial transformation

$$P(A) \approx CA^{-1}$$

thereby producing the desired solution state up to normalization. This implementation represents the most modern quantum linear-system solver investigated in the project and eliminates the eigenvalue quantization errors inherent in QPE-based approaches.

In the standard QSVT framework, one synthesizes a polynomial approximating the inverse function $f(\nu) = C/\nu$ on the spectrum of the block-encoded matrix. Our implementation instead uses the cosine block encoding $\cos(tA)$, so the synthesized polynomial approximates the transformed function $f(y) = \dfrac{t}{\arccos(y)}$, where $y = \cos(t\nu)$.

3.6 Readout Procedures

Entanglement with the Phase Register:
For the QPE-based implementations, *two* postselection steps are required. The first is the standard HHL postselection on the ancilla qubit used for eigenvalue inversion. The second projects the QPE phase register onto the all-zero state after inverse QPE. Because finite-precision phase estimation introduces eigenvalue quantization errors, inverse QPE does not perfectly disentangle the phase register from the system register. Retaining only the $|00\cdots0\rangle$ outcome removes these residual error components.

Measurement Strategies:
For the two use cases we stated earlier (top-k nodes, and risk score for a specified node), the readout procedure is simple. For top-k nodes, we simply sample all computational basis states. For predicting the score of a specified node we could also construct a simple circuit as shown in Figure 10 fo measure the predicted risk score of a single node. This observable estimates $|\langle\eta|x\rangle|^2$, from which the normalized risk score of the target node $|\eta\rangle$ can be inferred.

4 Results and Discussion

All of our experiments were based on statevector simulators. We ran our experiments on small and medium sized matrices to focus more on getting the prototype correct. At the same time, we also designed architectures with precision so we could repeat experiments at scale with ease. Our experiments verified correctness of multiple HHL and QSVT implementations (ranging from toy educational ones to larger implementations). Table 1 shows the comparison on various aspects between different variants we implemented with Model 2 with the configuration $N = 32$, sparsity

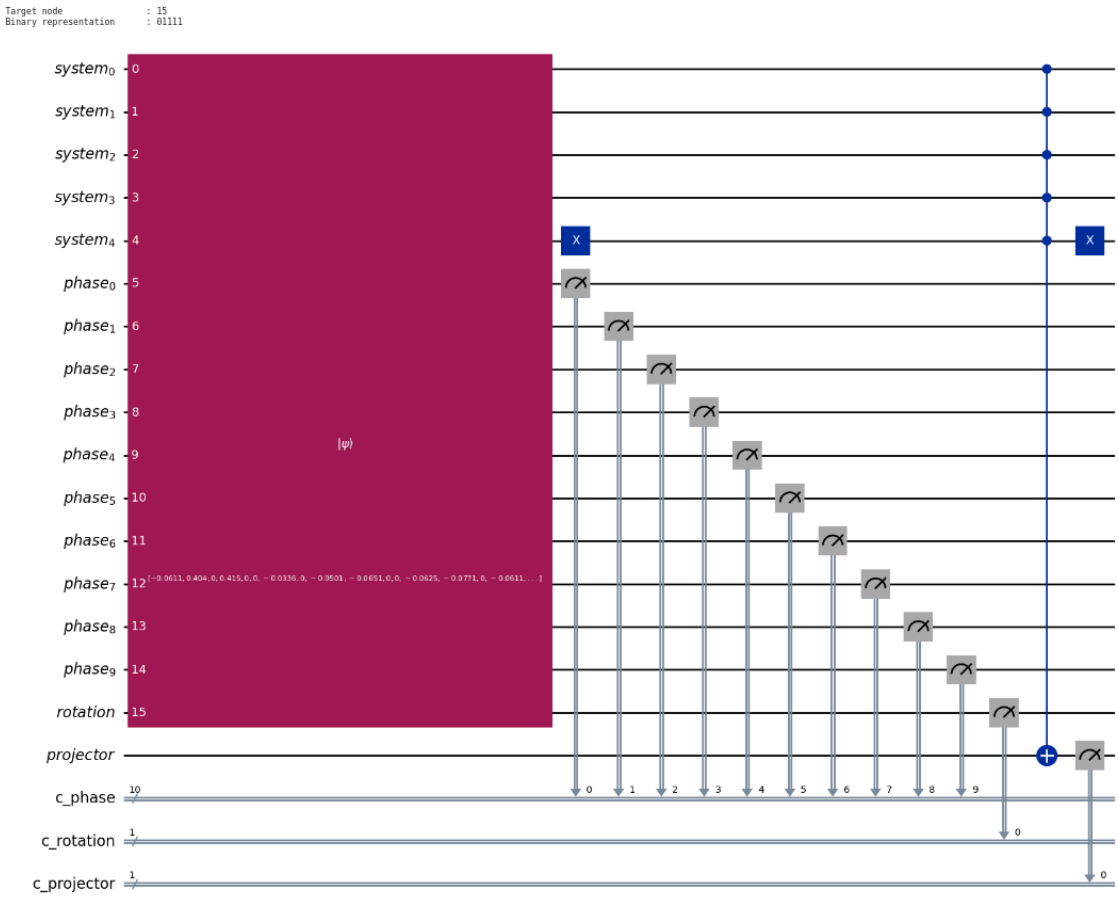

Figure 10: Measurement circuit for single node risk score for node $|15\rangle$

$s = 4$, and the diffusion parameter $\alpha = 0.6$. Here *fidelity* refers to the quantity $|\langle x|x_c\rangle|^2$ where x_c denotes the normalized expected classical solution vector, and $|x_c\rangle$ the corresponding quantum state obtained by amplitude encoding x_c. Also m denotes the number of qubits used in the phase (eigenvalue) register of QPE, d_{qpe} and d_{qsvt} denote the degree of the respective polynomial used in the approximation, D_{BC} and D_{LCU} denote the respective circuit depths of the block encoding circuits of the Berry-Childs block encoding and the LCU block encoding we used. Finally, ϵ denotes the desired overall solution accuracy. For the QPE-based implementations, the accuracy is dominated by QPE precision and thus an ϵ-accuracy is achieved by choosing $2^m = \Theta(\frac{1}{\epsilon})$. For the QSVT implementation ϵ denotes the maximum polynomial approximation error, and achieving this accuracy requires d_{qsvt} to be $O(\log(\frac{1}{\epsilon}))$.

Table 1: Comparison of the three quantum linear-system implementations (Model 2, $\alpha = 0.6$).

Metric	**QPE Variant 1**	**QPE Variant 2**	**QSVT**
Fidelity	0.998718	0.998465	0.999966
Post Selection Prob	0.263	0.261	0.381
Block Enc Oracle Calls	$2(2^m - 1)$	$2(2^m - 1)$	d_{qsvt}
Requires Poly Oracle	Yes	No	No
Circuit Depth Estimate	$O(2^m D_{BC} + d_{qpe})$	$O(2^m D_{BC} + 2^m)$	$O(d_{qsvt} D_{LCU})$
Registers used	Index, System, Row, Prepare, Phase, Poly, Rotation	Index, System, Row, Prepare, Phase, Rotation	Index, System, Row, Prepare
Total qubits	34	27	10
Requires QPE	Yes	Yes	No
Controlled rotations	Simple controlled rotations + Block Encoding	2^m multicontrolled rotations + Block Encoding	Block Encoding only
Postselection	Ancilla + Phase register	Ancilla + Phase register	Block-encoding ancillas
Asymptotic complexity	$O\big(\frac{\kappa s}{\epsilon}\,\mathrm{polylog}(N)\big)$	$O\big(\frac{\kappa s}{\epsilon}\,\mathrm{polylog}(N)\big)$	$O(\kappa s\,\log(1/\epsilon)\,\mathrm{polylog}(N))$

Here are the main inferences from the table.

1. **All three implementations produced highly accurate solutions.** The fidelities exceed 0.998 in every case, with the QSVT implementation achieving the highest fidelity. This demonstrates that both QPE-based and QSVT-based approaches successfully solve the graph-based financial diffusion problem for the tested instances.

2. **Variant 1 trades qubits for simpler circuit synthesis.** By explicitly basis-encoding the polynomial approximation, Variant 1 requires additional registers and qubits but only simple controlled rotations during eigenvalue inversion.

3. **Variant 2 reduces qubit requirements at the expense of circuit depth.** Eliminating the polynomial register significantly decreases the qubit count, but this is achieved by

replacing the polynomial oracle with an exponential network of multicontrolled rotations, making coherent implementation substantially more challenging.

4. **QSVT provides the most resource-efficient modern architecture.** It completely eliminates Quantum Phase Estimation, the phase register, polynomial evaluation, controlled rotations for eigenvalue inversion, and associated postselection on the phase register. Consequently, it requires the fewest qubits and exhibits the most favorable asymptotic scaling among the implemented approaches.

5. **The comparison illustrates the historical evolution of quantum linear-system algorithms.** The implemented architectures progress naturally from QPE-based HHL methods toward modern block-encoding and QSVT techniques, demonstrating how newer algorithms reduce resource requirements while improving asymptotic performance.

5 Representative Complexity Comparison with Classical Solutions

In this brief section we compare the best performing classical solvers for our problem with our quantum linear solvers.

1. **Classical nearly-linear Laplacian solvers remain the strongest classical baseline** for sparse graph-Laplacian systems, achieving nearly-linear work complexity and excellent practical scalability [KS16]. They therefore provide the most appropriate benchmark for evaluating quantum linear-system algorithms.

2. **Graph diffusion methods and Graph Neural Networks address different objectives.** Diffusion algorithms iteratively propagate information over graphs, while GNNs learn predictive models from labelled data. Although highly successful for graph analytics ([ACL06], [KW17]), neither directly solves the sparse matrix inversion problem considered in this project.

3. **Modern Quantum Linear System Algorithms offer a fundamentally different computational paradigm.** Under the standard assumptions of sparse matrices, bounded condition number, and efficient state preparation, block encoding and QSVT reduce the dependence on the system dimension from nearly linear to polylogarithmic while eliminating the explicit Quantum Phase Estimation used in earlier HHL-based approaches. This progression motivated the evolution of the implementations developed in this project.

Table 2 below shows the complexity comparison between the best known classical and quantum approaches. Here N denotes the number of graph nodes, E the number of graph edges, s the sparsity of the system matrix, κ the condition number, L the number of GNN message-passing layers, and ϵ the desired solution accuracy.

Note: For sparse graphs, the number of edges satisfies $E = O(N)$ (and may be larger depending on the graph density). Consequently, the classical methods listed above exhibit at least linear dependence on the graph size through E. Also, the complexity shown for Graph Neural Networks corresponds to **inference** using a trained model, which is approximately $O(EL)$, where L is the number of message-passing layers; training is substantially more expensive and additionally depends on the number of optimization epochs and the learning procedure.

Table 2: Representative complexity comparison of classical and quantum approaches for graph-based financial risk diffusion.

Method	**Representative Complexity**
Preconditioned Conjugate Gradient (PCG)	$O(E\sqrt{\kappa}\log(1/\epsilon))$
Nearly-linear Laplacian Solver	$O(E\,\mathrm{polylog}(N)\,\log(1/\epsilon))$
Graph Diffusion (e.g., Personalized PageRank)	$O(E\log(1/\epsilon))$ (iterative propagation)
Graph Neural Networks (Inference)	Approx $O(EL)$ for L message-passing layers
QPE-based HHL (implemented)	$O\Big(\frac{\kappa s}{\epsilon}\,\mathrm{polylog}(N)\Big)$
QSVT-based QLSA	$O(\kappa s\log(1/\epsilon)\,\mathrm{polylog}(N))$

6 BQP Completeness of Matrix Inversion

In this brief section, we will present a brief proof of BQP completeness of the matrix inversion problem [HHL09]. The matrix inversion problem is **BQP-complete**, meaning that it is both solvable by a quantum computer in polynomial time and sufficiently powerful to simulate any problem in the complexity class BQP. Here is the proof.

1. **Matrix inversion belongs to BQP.** Quantum Linear System Algorithms (QLSAs), including HHL and modern QSVT-based methods, prepare a quantum state proportional to the solution of a sparse linear system,

   $$Ax = b$$

   in time polynomial in the sparsity s, condition number κ, and $\log N$, under the standard assumptions of efficient state preparation and Hamiltonian simulation.

2. **Any quantum circuit can be encoded as a sparse linear system.** Given an arbitrary polynomial-size quantum circuit U, one can construct, in polynomial time, a sparse, well-conditioned matrix A together with a vector b such that the solution

   $$x = A^{-1}b$$

   encodes the complete computation history of the circuit (Feynman–Kitaev history state construction).

3. **Recovering the solution reproduces the circuit output.** Measuring suitable observables of the quantum state proportional to $A^{-1}b$ yields the acceptance probability of the original quantum circuit. Consequently, any problem solvable by a polynomial-time quantum algorithm can be reduced to an instance of sparse matrix inversion.

4. **Therefore, matrix inversion is BQP-complete.** Since sparse matrix inversion can itself be solved efficiently on a quantum computer (membership in BQP) and every BQP computation can be reduced to sparse matrix inversion (BQP-hardness), the problem is BQP-complete.

7 Conclusion

This project investigated Quantum Linear System Algorithms (QLSAs) for graph-based financial risk diffusion, beginning with the mathematical formulation of graph-Laplacian diffusion

models and progressing through every major component of the quantum algorithmic pipeline: sparse state preparation, Hamiltonian simulation, block encoding, eigenvalue inversion, and efficient readout. Multiple implementations were developed, including two Berry–Childs/QPE-based HHL variants employing different eigenvalue-inversion architectures and a modern LCU-QSVT solver based on block encoding and Quantum Singular Value Transformation. All implementations were validated against classical solutions, achieving high fidelities while illustrating the trade-offs between qubit count, circuit depth, oracle complexity, and asymptotic scaling.

Beyond the individual implementations, the project demonstrates the historical evolution of quantum linear-system algorithms from traditional QPE-based HHL methods to modern block-encoding and QSVT techniques. While current fault-tolerant quantum hardware remains unavailable, the study highlights how successive algorithmic advances eliminate major bottlenecks such as explicit Quantum Phase Estimation, controlled eigenvalue rotations, and unfavorable precision scaling. Together with the comparison against the best-known classical approaches and the BQP-completeness of sparse matrix inversion, these results establish graph-based financial risk diffusion as a compelling application of quantum linear-system algorithms and provide a solid foundation for future large-scale implementations on fault-tolerant quantum computers.

References

[ACL06] Reid Andersen, Fan Chung, and Kevin Lang, *Local graph partitioning using pagerank vectors*, Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS), 2006, pp. 475–486.

[BC12] Dominic W. Berry and Andrew M. Childs, *Black-box hamiltonian simulation and unitary implementation*, Quantum Information and Computation **12** (2012), no. 1–2, 29–62.

[BCC+15] Dominic W. Berry, Andrew M. Childs, Richard Cleve, Robin Kothari, and Rolando D. Somma, *Simulating hamiltonian dynamics with a truncated taylor series*, Physical Review Letters **114** (2015), 090502.

[BMN04] Mikhail Belkin, Irina Matveeva, and Partha Niyogi, *Regularization and semi-supervised learning on large graphs*, International Conference on Computational Learning Theory (COLT), 2004, pp. 624–638.

[CW12] Andrew M. Childs and Nathan Wiebe, *Hamiltonian simulation using linear combinations of unitary operations*, Quantum Information and Computation **12** (2012), no. 11–12, 901–924.

[GH21] Niels Gleinig and Torsten Hoefler, *An efficient algorithm for sparse quantum state preparation*, Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC), IEEE, 2021, pp. 433–438.

[GSLW19] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe, *Quantum singular value transformation and beyond: Exponential improvements for quantum matrix arithmetics*, Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC), Association for Computing Machinery, 2019, pp. 193–204.

[HHL09] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd, *Quantum algorithm for linear systems of equations*, Physical Review Letters **103** (2009), no. 15.

[KS16] Rasmus Kyng and Sushant Sachdeva, *Approximate gaussian elimination for laplacians: Fast, sparse, and simple*, Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science (FOCS), 2016, pp. 573–582.

[KW17] Thomas N. Kipf and Max Welling, *Semi-supervised classification with graph convolutional networks*, International Conference on Learning Representations (ICLR), 2017.

[LC16] Guang Hao Low and Isaac L. Chuang, *Hamiltonian simulation by qubitization*, arXiv preprint arXiv:1610.06546 (2016).

[LC19] ———, *Hamiltonian simulation by qubitization*, Quantum **3** (2019), 163.

[SV24] Alok Shukla and Prakash Vedula, *An efficient quantum algorithm for preparation of uniform quantum superposition states*, arXiv preprint arXiv:2306.11747 (2024).

[ZBL+04] Dengyong Zhou, Olivier Bousquet, Thomas N. Lal, Jason Weston, and Bernhard Schölkopf, *Learning with local and global consistency*, Advances in Neural Information Processing Systems, vol. 16, 2004.

[ZGL03] Xiaojin Zhu, Zoubin Ghahramani, and John D. Lafferty, *Semi-supervised learning using gaussian fields and harmonic functions*, Proceedings of the 20th International Conference on Machine Learning (ICML), 2003, pp. 912–919.